



ECOLE POLYTECHNIQUE DE LOUVAIN

Pipeline description

Mémoire : Video alignment in a railway infrastructure

Julian GOSSELIN - 20992100 - SINF

Table des matières

1	Phase 1 : L'Alignement Hybride (Approche <i>Coarse-to-Fine</i>)	1
1.1	Étape 1.1 : Macro-synchronisation (Le Squelette DTW)	1
1.2	Étape 1.2 : Micro-alignement	1
1.3	Étape 1.3 : Optimisation Globale et Contrainte de Monotonie	3
1.4	Étape 1.4 : Lissage Post-traitement (<i>Anti-Jitter</i>)	3
2	Phase 2 : Calcul des Métriques d'Évaluation	3
2.1	Répartition des Sources (Taux de réussite)	3
2.2	Évaluation de la Cohérence Temporelle	4
2.3	Analyse de la Vitesse Relative	4
2.4	Qualité Géométrique et Correction du Squelette	4

1 Phase 1 : L'Alignement Hybride (Approche *Coarse-to-Fine*)

La première phase de notre pipeline constitue le cœur de l'algorithme d'alignement. Elle repose sur une architecture dite *Coarse-to-Fine* qui combine la robustesse de l'analyse globale du mouvement avec la précision de l'appariement de caractéristiques locales. Cette phase est divisée en quatre sous-étapes majeures.

1.1 Étape 1.1 : Macro-synchronisation (Le Squelette DTW)

L'objectif de cette étape est de créer un chemin temporel approximatif mais globalement robuste, agissant comme un "squelette" pour empêcher toute dérive à long terme de l'alignement.

- **Extraction des caractéristiques par flux optique** : Plutôt que de comparer les pixels bruts, l'algorithme extrait une signature du mouvement global pour chaque *frame*. Il utilise l'algorithme de flux optique dense de Farneback pour mesurer le champ de déplacement des pixels. Pour optimiser le temps de calcul et filtrer le bruit environnemental, l'image est d'abord redimensionnée, le tiers supérieur (le ciel) est ignoré, et les *frames* sont sous-échantillonnées (par exemple, une image sur cinq). Enfin, l'image est divisée en trois régions verticales (gauche, centre, droite) pour extraire des statistiques de mouvement (moyenne et écart-type), capturant ainsi l'effet de parallaxe propre au déplacement du train.
- **Alignement par *Dynamic Time Warping* (DTW)** : L'algorithme DTW est ensuite utilisé pour synchroniser ces deux séquences temporelles de mouvement. Son fonctionnement interne se divise en plusieurs phases mathématiques clés :
 - *Matrice de distances* : Une matrice de distance euclidienne est calculée entre le vecteur de caractéristiques de chaque *frame* i de la vidéo 1 et celui de chaque *frame* j de la vidéo 2.
 - *Programmation dynamique et matrice de coût* : L'algorithme construit une matrice de coût cumulé afin de trouver le "chemin de déformation" (*warping path*) optimal. Ce chemin relie les indices des deux vidéos en minimisant la distance globale cumulée depuis le point de départ. Les déplacements au sein de cette matrice sont contraints par la monotonie temporelle : le temps ne peut qu'avancer, ce qui limite les déplacements aux mouvements diagonaux, vers la droite ou vers le bas.
 - *Pénalité de pas (Step Penalty)* : Afin de s'adapter à la dynamique fluide et continue d'un train en mouvement, une pénalité adaptative est ajoutée mathématiquement aux mouvements non diagonaux (qui correspondent à des insertions ou des suppressions temporelles). Cela empêche l'algorithme de "bégayer" (c'est-à-dire d'associer une seule *frame* statique d'une vidéo à de multiples *frames* de l'autre) et favorise fortement une synchronisation continue de type 1:1.
 - *Variante Open-End* : Le DTW classique force le chemin à se terminer exactement dans le coin inférieur droit de la matrice de coût, ce qui obligerait la fin des deux séquences vidéo à correspondre parfaitement. Notre implémentation utilise une variante *Open-End* qui cherche la sortie optimale (le coût minimum) sur la dernière ligne ou la dernière colonne de la matrice. Cela autorise les vidéos à se terminer à des points géographiques différents, respectant la réalité physique de notre jeu de données ferroviaires où les trains peuvent parcourir des distances différentes pour une même durée d'enregistrement.
- **Cartographie d'interpolation** : Le chemin d'alignement brut issu du DTW est finalement transformé en un dictionnaire de correspondance fluide. Il fournit pour chaque *frame* de la vidéo 1, une prédiction de l'index de la *frame* correspondante dans la vidéo 2. Les *frames* intermédiaires manquantes (dues au sous-échantillonnage initial de l'étape d'extraction) sont déduites par interpolation linéaire entre les points d'ancrage validés par le chemin DTW.

1.2 Étape 1.2 : Micro-alignement

Cette étape affine la prédiction macroscopique du DTW pour obtenir une correspondance exacte au pixel près. Pour ce faire, la pipeline s'appuie sur des algorithmes d'extraction et de description de caractéristiques locales (*Feature Matching*). Le système supporte trois algorithmes distincts, offrant différents compromis entre vitesse et précision :

- **ORB (*Oriented FAST and Rotated BRIEF*)** : Proposé comme une alternative très rapide à

SIFT, ORB combine le détecteur de coins FAST (auquel il ajoute une notion d'orientation) et le descripteur binaire BRIEF. Bien qu'il soit le plus rapide des trois et invariant à la rotation, il est le plus sensible au bruit et gère moins bien les changements d'échelle importants.

- **BRISK (*Binary Robust Invariant Scalable Keypoints*)** : Cet algorithme détecte les points d'intérêt dans un espace d'échelle continu en utilisant une pyramide d'images et un détecteur AGAST. Son descripteur binaire utilise un motif d'échantillonnage circulaire inspiré de la rétine humaine (plus dense au centre, plus espacé en périphérie). Il offre une excellente robustesse aux changements d'échelle (zoom) et au flou.
- **AKAZE (*Accelerated-KAZE*)** : Contrairement aux autres méthodes qui utilisent un flou Gaussien (lissant toute l'image uniformément), AKAZE travaille dans un espace d'échelle non-linéaire (diffusion explicite rapide). Il permet de réduire le bruit (comme le ballast des voies) tout en préservant parfaitement les arêtes et contours structurés (les rails, les poteaux). Son descripteur binaire M-LDB capture la structure locale avec une très haute précision.

Une fois l'algorithme choisi, le processus de micro-alignement s'articule autour d'une série de filtres et d'optimisations très stricts pour éviter les fausses correspondances :

- **Fenêtre de recherche restreinte** : Pour une *frame* donnée de la vidéo 1, l'algorithme cherche la meilleure correspondance dans la vidéo 2 uniquement au sein d'une fenêtre très étroite centrée sur la prédiction du DTW (ex. ± 10 *frames*). Cela réduit drastiquement les temps de calcul et empêche d'associer des éléments lointains.
- **Extraction et appariement binaire (Test de Lowe)** : Lorsqu'un point d'intérêt est détecté sur une image, l'algorithme lui attribue une « signature » locale pour permettre sa ré-identification dans l'image suivante : c'est ce qu'on appelle un *descripteur*. Les trois algorithmes implémentés dans le code de cette pipeline (ORB, BRISK et AKAZE) ont l'avantage de générer nativement des descripteurs *binaires*, c'est-à-dire que cette signature est une simple séquence de zéros et de uns.

Pour comparer et appairer les points entre la vidéo 1 et la vidéo 2, le système emploie un algorithme de force brute (*Brute Force Matcher*) basé sur la **distance de Hamming**. Cette distance mesure la dissimilitude en comptant très rapidement, via une opération XOR, le nombre de bits différents entre deux signatures.

Toutefois, l'environnement ferroviaire regorge de motifs répétitifs (cailloux du ballast, traverses de chemin de fer quasi-identiques) qui trompent facilement l'ordinateur. Pour contrer ce bruit visuel, le système extrait pour chaque point les *deux* meilleures correspondances probables dans l'image cible. Le **test de ratio de Lowe** est ensuite appliqué : une correspondance n'est conservée que si la distance du meilleur candidat (D_1) est significativement inférieure à celle du second (D_2), selon une condition stricte (ici, $D_1 < 0.75 \times D_2$). Ce filtrage est crucial, car il permet de rejeter massivement les associations ambiguës pour ne garder que les points dont la fiabilité est mathématiquement incontestable.

- **Filtre de cohérence spatiale (Effet de Parallaxe)** : L'algorithme divise chaque image géométriquement en deux (moitié gauche et moitié droite). Il impose qu'une caractéristique détectée à gauche dans la vidéo 1 doive impérativement correspondre à une caractéristique située à gauche dans la vidéo 2 (et inversement pour la droite). Cela filtre les appariements physiquement impossibles.
- **Validation Géométrique (Homographie et Échelle)** : Les correspondances restantes sont soumises à l'algorithme RANSAC pour calculer une matrice d'homographie et compter le nombre d'*inliers* (points géométriquement valides). Une optimisation supplémentaire vérifie les échelles extraites de cette homographie : pour des vidéos de voies ferrées, l'échelle temporelle instantanée doit être proche de 1.0. Si la transformation implique une déformation extrême (échelle < 0.7 ou > 1.3), le score d'*inliers* est lourdement pénalisé, car cela traduit un faux positif.
- **Pénalité de distance (Scoring)** : Pour éviter d'associer des motifs répétitifs aberrants (comme deux traverses de chemin de fer distinctes qui se ressemblent), le score brut d'*inliers* est pondéré par une fonction de décroissance exponentielle basée sur la distance temporelle par rapport à la prédiction DTW :

$$Score_{pondéré} = Inliers \times e^{-k \cdot |\Delta|} \quad (1)$$

où Δ est la distance entre la *frame* candidate et la prédiction DTW, et k est une constante de

pénalité de distance (ex : 0.15). Les K meilleurs candidats (*Top-K*) sont conservés pour chaque image pour la phase d'optimisation globale.

1.3 Étape 1.3 : Optimisation Globale et Contrainte de Monotonie

Le micro-alignement provoque des résultats assez précis, cependant elles peuvent amener à des tremblements dans le résultat final. (C'est la partie dont la logique peut-être améliorée. Actuellement la logique est assez simpliste et pourrait fonctionner un peu mieux).

- **Programmation dynamique** : L'algorithme cherche le chemin optimal à travers la matrice des K meilleurs candidats pour l'ensemble de la vidéo.
- **Contrainte stricte** : Le chemin doit être chronologiquement monotone, c'est-à-dire que l'index de la vidéo 2 ne peut jamais décroître ($V2_i \geq V2_{i-1}$).
- **Filet de sécurité (*Fallback*)** : Si aucune correspondance locale ne satisfait les conditions minimales (par exemple si l'image est noire dans un tunnel et qu'il y a moins de 4 *inliers*), l'algorithme abandonne la recherche visuelle locale et adopte la prédiction du DTW par défaut comme vérité, garantissant l'absence de perte de suivi.

1.4 Étape 1.4 : Lissage Post-traitement (*Anti-Jitter*)

Bien que l'optimisation globale (Étape 1.3) garantisse mathématiquement l'absence de retours temporels, ses prédictions sont par nature discrètes (basées sur des entiers). Cette rigidité peut générer un effet de balbutiement visuel (*jitter*), où l'image cible "stagne" pendant quelques *frames* avant de "sauter" brusquement pour rattraper son retard. Afin de garantir une progression fluide et cinématique lors de la superposition finale des vidéos, une ultime passe de lissage est appliquée.

Contrairement à l'étape précédente qui est axée sur la **sélection** absolue d'une *frame*, cette étape agit comme un filtre de **correction esthétique** sur les résultats déjà validés :

- **Filtre par moyenne mobile pondérée** : L'algorithme déploie une fenêtre glissante (ex : 5 *frames*) sur la ligne temporelle. Pour chaque position, il ne conserve pas la valeur brute, mais calcule la moyenne des positions environnantes.
- **Pondération par la confiance** : L'intelligence de ce filtre réside dans l'intégration de la métrique de qualité. Les positions environnantes ne valent pas toutes le même poids : une *frame* validée par un score spectaculaire d'*inliers* *RANSAC* aura une force d'attraction massive sur la moyenne, tandis qu'une correspondance incertaine sera lissée au profit de ses voisines robustes.
- **Validation stricte de redressement** : L'application d'une moyenne lissante sur une courbe pouvant induire de légères oscillations, une dernière passe algorithmique vérifie et corrige chaque paire finale pour s'assurer que le lissage n'a mathématiquement et physiquement généré aucun retour en arrière temporel (respect final de la contrainte $V2_i \geq V2_{i-1}$).

2 Phase 2 : Calcul des Métriques d'Évaluation

Une fois le processus terminé, voici tout les outils générés afin de me faire une idée précise de comment l'algorithme a fonctionné, s'il est confiant, ...

2.1 Répartition des Sources (Taux de réussite)

Cette métrique évalue la proportion de *frames* alignées par la méthode locale (AKAZE) par rapport à celles alignées par le filet de sécurité (DTW).

- **Pourcentage *AKAZE Refined*** : Indique le taux de *frames* où l'algorithme a pu trouver suffisamment de points d'intérêt (*inliers* ≥ 4) pour garantir un alignement géométrique parfait au pixel près. Un score élevé traduit de bonnes conditions visuelles.
- **Pourcentage *DTW Fallback*** : Représente les zones où l'image était trop dégradée (tunnel, vitesse excessive) et où le système s'est replié sur la macro-synchronisation du flux optique pour éviter de perdre le suivi.

2.2 Évaluation de la Cohérence Temporelle

L'alignement de vidéos ferroviaires requiert une fluidité temporelle parfaite pour éviter les "sauts" lors de la lecture simultanée.

- **Violations de monotonie** : Le système compte le nombre exact de fois où l'index de la vidéo 2 recule par rapport à la *frame* précédente ($V2_i < V2_{i-1}$).
- **Score de monotonie** : Exprimé en pourcentage, il quantifie le respect strict de la chronologie. Grâce à l'optimisation globale de l'étape 1.3, ce score est conçu pour tendre vers 100%.

2.3 Analyse de la Vitesse Relative

Étant donné que les trains ne circulent pas exactement à la même vitesse le jour J et le jour J-1, l'algorithme calcule la dynamique des vitesses relatives tout au long du trajet.

- **Vitesse moyenne et écart-type** : La vitesse instantanée est calculée comme le ratio de la progression dans la vidéo 2 par rapport à la vidéo 1 ($\frac{\Delta V2}{\Delta V1}$). La moyenne indique quel train était globalement le plus rapide, tandis que l'écart-type met en évidence les variations d'allure (ex : freinages différents).

2.4 Qualité Géométrique et Correction du Squelette

Enfin, la pipeline évalue la précision pure du *Feature Matching* et son utilité par rapport au simple flux optique.

- **Moyenne des Inliers AKAZE** : Évalue la force des correspondances trouvées. Un nombre moyen d'inliers élevé garantit que la matrice d'homographie calculée par RANSAC est extrêmement solide.
- **Correction moyenne (en *frames*)** : Mesure l'écart exact entre la prédiction initiale fournie par le DTW et la position finale corrigée par AKAZE ($|V2_{final} - V2_{DTW}|$). Cela permet de chiffrer précisément l'apport de la méthode.